

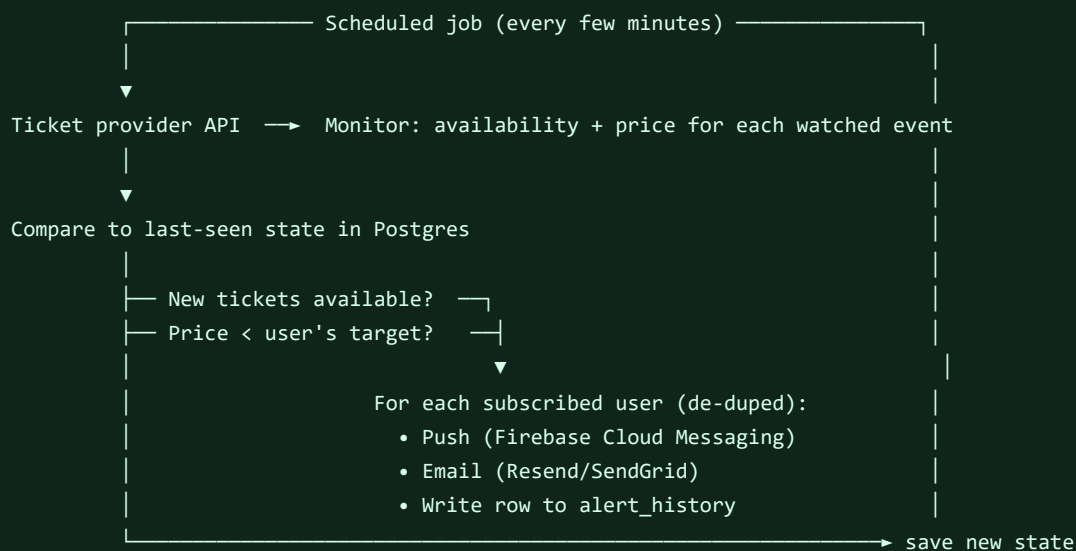


Ticket Monitoring Bot — Full Spec & Build Plan

A web app that **monitors ticket events**, tracks **availability and price**, and sends users **instant email + push notifications** when (a) new tickets appear or (b) the price drops below their target.

✅ **This is a monitoring & alerting service.** It reads public availability via official APIs and notifies humans. ❌ **It does NOT auto-buy, auto-checkout, or create accounts to bypass limits.** Those are illegal (US BOTS Act, Ticketmaster/FIFA terms) and are intentionally excluded.

1. What it does (one picture)



2. Tech stack

Layer	Choice	Notes
Framework	Next.js (App Router) + TypeScript	UI + API routes in one app
Database	PostgreSQL (Neon)	Already provisioned on Vercel
ORM	Drizzle ORM	Type-safe schema + migrations
Auth	Auth.js (NextAuth v5)	Email/password + Google OAuth
Push	Firebase Cloud Messaging (FCM)	Cross-platform (web + Android/iOS)
Email	Resend (or SendGrid)	Transactional alert emails
Ticket data	Ticketmaster Discovery API (+ SeatGeek for resale prices)	Availability + price
Scheduler	cron-job.org (free) or Vercel Cron (Pro)	Pings the monitor endpoint

Layer	Choice	Notes
Hosting	Vercel	HTTPS required for push
Monitoring	Sentry (optional)	Catch failed jobs

3. Architecture

```

app/
  (auth)/login, /register      → Auth.js pages
  dashboard/
    page.tsx                  → overview
    events/                   → browse + add events to watchlist
    alerts/                   → alert history
    settings/                 → notification prefs, target prices
  api/
    auth/[...nextauth]/route.ts → Auth.js handler
    watchlist/route.ts        → add/remove watched events (POST/DELETE)
    subscriptions/route.ts    → register FCM token / push subscription
    monitor/route.ts          → THE scheduled job (cron hits this)
    events/route.ts           → search/list provider events

lib/
  db/
    schema.ts                 → Drizzle tables (section 4)
    index.ts                  → drizzle client (Neon)
  providers/
    ticketmaster.ts           → availability + priceRanges
    seatgeek.ts               → resale prices (optional)
  notify/
    push.ts                   → FCM send
    email.ts                  → Resend send
    monitor.ts                → core: detect changes, fan out alerts
    auth.ts                   → Auth.js config

drizzle/                      → generated migrations

```

4. Database schema (Drizzle)

```

// lib/db/schema.ts
import { pgTable, serial, text, integer, boolean, timestamp, numeric, uniqueIndex } from "drizzle-orm/pg-core"

export const users = pgTable("users", {
  id: text("id").primaryKey(), // from Auth.js
  email: text("email").notNull().unique(),
  name: text("name"),
  createdAt: timestamp("created_at").defaultNow(),
});

export const events = pgTable("events", {

```

```

    id: text("id").primaryKey(),           // provider event id
    name: text("name").notNull(),
    venue: text("venue"),
    city: text("city"),
    startsAt: timestamp("starts_at"),
    url: text("url"),                       // official buy page
    lastStatus: text("last_status"),        // AVAILABLE | SOLD_OUT
    lastMinPrice: numeric("last_min_price"), // last seen lowest price
    updatedAt: timestamp("updated_at").defaultNow(),
  });

// One row per (user, event) – the watchlist entry, with their target price.
export const watchlist = pgTable("watchlist", {
  id: serial("id").primaryKey(),
  userId: text("user_id").notNull().references(() => users.id),
  eventId: text("event_id").notNull().references(() => events.id),
  targetPrice: numeric("target_price"),    // null = "any availability"
  active: boolean("active").default(true),
  createdAt: timestamp("created_at").defaultNow(),
}, (t) => ({ uniq: uniqueIndex("uniq_user_event").on(t.userId, t.eventId) }));

// Devices/tokens to push to (a user can have several).
export const pushTokens = pgTable("push_tokens", {
  id: serial("id").primaryKey(),
  userId: text("user_id").notNull().references(() => users.id),
  fcmToken: text("fcm_token").notNull().unique(),
  createdAt: timestamp("created_at").defaultNow(),
});

// Alert history – also used to prevent duplicate alerts.
export const alertHistory = pgTable("alert_history", {
  id: serial("id").primaryKey(),
  userId: text("user_id").notNull().references(() => users.id),
  eventId: text("event_id").notNull().references(() => events.id),
  type: text("type").notNull(),           // NEW_AVAILABLE | PRICE_DROP
  price: numeric("price"),
  channel: text("channel"),               // push | email
  sentAt: timestamp("sent_at").defaultNow(),
});

```

5. Monitoring workflow (the core logic)

`lib/monitor.ts`, called by `GET/POST /api/monitor` (the cron target):

1. Load all DISTINCT events that have ≥ 1 active watchlist entry.
2. For each event, call the provider API $\rightarrow$ { status, minPrice }.
3. Determine triggers vs the event's last-seen state:
 - NEW_AVAILABLE : lastStatus == SOLD_OUT && status == AVAILABLE
 - PRICE_DROP : minPrice < watchlist.targetPrice (per user)
4. For each triggered (user, event):
 - De-dupe: skip if an alert of same type for same event was sent to this user within the cooldown window (e.g. last 60 min) – check alert_history.
 - Send Push (FCM) + Email (Resend).

- Insert row(s) into alert_history.
5. Update events.lastStatus / lastMinPrice / updatedAt.

De-dup rule: never send the same (user, event, type) more than once per cooldown window. The alert_history table is the source of truth.

6. Notifications

Push — Firebase Cloud Messaging

- Client: init Firebase, request permission, get **FCM token**, POST it to /api/subscriptions → stored in push_tokens .
- Server: lib/notify/push.ts uses the **firebase-admin** SDK to send to tokens.
- Dead tokens (FCM returns messaging/regISTRATION-TOKEN-NOT-REGISTERED) are deleted from push_tokens .

Email — Resend

- lib/notify/email.ts sends a templated email: event name, trigger reason (new tickets / price dropped to \$X), and a "Buy on official site" button.

7. Authentication (Auth.js / NextAuth v5)

- Providers: **Credentials (email+password)** and **Google OAuth**.
- Sessions in Postgres (Drizzle adapter).
- All dashboard/* pages + write APIs require a session.
- Each user only sees/edits **their own** watchlist and alerts.

8. Dashboard (protected UI)

- **Overview** — watched events with current status + price + last alert.
- **Events** — search the provider, click + **Watch**, set a **target price**.
- **Alerts** — history table (event, type, price, time, channel).
- **Settings** — toggle push/email, manage devices, default target price.

9. Build phases (ship incrementally)

Phase	Deliverable	Done when
0	Migrate current app → TypeScript + Drizzle schema	Build green, existing alerts still work
1	Auth (login/register, Google)	A user can sign up and log in
2	Watchlist (add/remove events, per-user)	Users watch their own events
3	Monitor rewrite for per-user + price	/api/monitor triggers per watchlist
4	FCM push	Real push to a registered device

Phase	Deliverable	Done when
5	Email (Resend)	Alert email arrives
6	Target-price alerts (+ SeatGeek resale price)	Price-drop alert fires
7	Dashboard polish + alert history + de-dupe	Full UI, no duplicate spam
8	Sentry + cron hardening + docs	Production-ready

10. Environment variables

```
# Database (Neon – auto-added by Vercel)
POSTGRES_URL=

# Auth.js
AUTH_SECRET=
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=

# Firebase Cloud Messaging (server)
FIREBASE_PROJECT_ID=
FIREBASE_CLIENT_EMAIL=
FIREBASE_PRIVATE_KEY=
# Firebase web config (client) – NEXT_PUBLIC_*
NEXT_PUBLIC_FIREBASE_API_KEY=
NEXT_PUBLIC_FIREBASE_PROJECT_ID=
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=
NEXT_PUBLIC_FIREBASE_APP_ID=
NEXT_PUBLIC_FIREBASE_VAPID_KEY=

# Email
RESEND_API_KEY=

# Ticket providers
TICKETMASTER_API_KEY=
SEATGEEK_CLIENT_ID=          # optional, for resale prices

# Cron protection
CRON_SECRET=                # require this in /api/monitor to block public abuse
```

11. Scheduling

- **Free:** cron-job.org pings `https://<app>/api/monitor?key=CRON_SECRET` every 5 min.
- **Pro:** Vercel Cron in `vercel.json`.
- `/api/monitor` checks `CRON_SECRET` so randoms can't trigger it.
- ⚠️ Respect provider limits: Ticketmaster ~5,000/day; batch events per run.

12. Legal / compliance (must keep)

- ☐ Read availability/price via official APIs and **alert only** — never buy.
 - ☐ No checkout automation, no multiple/fake accounts to dodge limits.
 - ☐ Alerts link to the **official purchase page**; the user buys manually.
 - ☐ Respect Ticketmaster/SeatGeek terms + rate limits.
 - ☐ Disclaimer: "Independent alert service. Not affiliated with FIFA/Ticketmaster."
 - ☐ Privacy policy (emails, tokens) + GDPR; ToS if charging money.
-

13. Definition of done (v1)

- ☐ User can register, log in, and manage a watchlist
 - ☐ User sets a target price per event
 - ☐ Scheduled monitor runs reliably and respects API limits
 - ☐ New-availability AND price-drop alerts fire via push + email
 - ☐ Alert history shown; duplicates suppressed within cooldown
 - ☐ Deployed on Vercel with Postgres; secrets in env vars
 - ☐ Legal disclaimer + privacy policy live
-

14. What already exists in this repo (reuse, don't rebuild)

- ☒ Next.js app + Vercel deploy + Neon Postgres
- ☒ Provider integration pattern (`lib/tickets.js` → Ticketmaster attraction query)
- ☒ Availability-change detection + de-dupe-on-change (`lib/check-tickets.js`)
- ☒ Push pipeline (currently Web Push/VAPID — to be swapped for FCM)
- ☒ Cron pattern (cron-job.org → `/api/check-tickets`)

This spec turns the working single-list alerter into a multi-user product with accounts, price tracking, email, and a dashboard.